

# AUTOPULSE

## Enterprise Management and Scheduling System for Auto Repair Shops

---

*Final Project Documentation*

### Development Team

Cozi David  
Dascălu Raul  
Fret Dario  
Gîrz Ionuț

|                |                      |
|----------------|----------------------|
| <b>Course:</b> | Software Engineering |
| <b>Date:</b>   | May 2026             |
| <b>Repo:</b>   | GitHub               |

# Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Project Description</b>                                      | <b>3</b>  |
| 1.1      | Problem Statement . . . . .                                     | 3         |
| 1.2      | The Solution . . . . .                                          | 3         |
| 1.3      | What Makes It Unique . . . . .                                  | 4         |
| <b>2</b> | <b>Team &amp; Contributions</b>                                 | <b>4</b>  |
| 2.1      | Team Composition and Roles Assignment . . . . .                 | 4         |
| 2.2      | Detailed Contributions . . . . .                                | 4         |
| 2.3      | Workflow and Collaboration Methodology . . . . .                | 6         |
| <b>3</b> | <b>Actors &amp; User Stories</b>                                | <b>6</b>  |
| 3.1      | Actors Definition . . . . .                                     | 6         |
| 3.2      | User Stories and Acceptance Criteria . . . . .                  | 7         |
| <b>4</b> | <b>UML Diagrams</b>                                             | <b>9</b>  |
| 4.1      | Use Case Diagram . . . . .                                      | 9         |
| 4.2      | Class Diagram . . . . .                                         | 10        |
| 4.3      | Sequence Diagram . . . . .                                      | 11        |
| <b>5</b> | <b>Architecture</b>                                             | <b>12</b> |
| 5.1      | System Architecture Overview . . . . .                          | 12        |
| 5.2      | Technology Stack Breakdown . . . . .                            | 12        |
| 5.3      | System Architecture Diagram . . . . .                           | 13        |
| 5.4      | Key Architectural Subsystems . . . . .                          | 14        |
| <b>6</b> | <b>Database Schema</b>                                          | <b>14</b> |
| 6.1      | Entity-Relationship Diagram (ERD) . . . . .                     | 14        |
| 6.2      | Schema Description and Relational Mapping . . . . .             | 15        |
| 6.2.1    | User Inheritance Hierarchy . . . . .                            | 15        |
| 6.2.2    | Core Operational Entities . . . . .                             | 15        |
| 6.2.3    | Technical Maintenance and Financial Ledger Entries . . . . .    | 16        |
| <b>7</b> | <b>API Documentation</b>                                        | <b>16</b> |
| 7.1      | API Architecture and Protocol Specifications . . . . .          | 16        |
| 7.1.1    | Global HTTP Status Code Matrix . . . . .                        | 16        |
| 7.1.2    | Security and Authentication Headers . . . . .                   | 17        |
| 7.2      | Comprehensive Endpoint Map . . . . .                            | 17        |
| 7.3      | Granular Endpoint Specifications and Payload Examples . . . . . | 17        |
| 7.3.1    | Authentication & Profile Module . . . . .                       | 17        |
| 7.3.2    | Automotive Asset Fleet Module . . . . .                         | 19        |
| 7.3.3    | Service Scheduling Module . . . . .                             | 20        |
| 7.3.4    | Technical Maintenance Financial Ledger Module . . . . .         | 21        |
| 7.3.5    | Human Resource Management Module . . . . .                      | 22        |
| 7.3.6    | System Metrics Administration Module . . . . .                  | 23        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>8</b>  | <b>Application Screenshots</b>                           | <b>23</b> |
| 8.1       | Public Interface and Authentication . . . . .            | 24        |
| 8.2       | Client Workspace . . . . .                               | 24        |
| 8.3       | Mechanic Task Management . . . . .                       | 25        |
| 8.4       | Managerial Dashboard . . . . .                           | 26        |
| 8.5       | Admin Dashboard . . . . .                                | 26        |
| <b>9</b>  | <b>Testing</b>                                           | <b>27</b> |
| 9.1       | Testing Strategy . . . . .                               | 27        |
| 9.2       | Tested Scenarios . . . . .                               | 28        |
| 9.3       | Results . . . . .                                        | 28        |
| <b>10</b> | <b>Security Measures</b>                                 | <b>29</b> |
| 10.1      | Authentication and Authorization . . . . .               | 29        |
| 10.2      | Concurrent Session and State Security . . . . .          | 29        |
| 10.3      | Network and API Protection . . . . .                     | 29        |
| <b>11</b> | <b>Setup Instructions</b>                                | <b>30</b> |
| 11.1      | Prerequisites . . . . .                                  | 30        |
| 11.2      | Environment and Database Configuration . . . . .         | 30        |
| 11.3      | Backend Initialization (Spring Boot) . . . . .           | 30        |
| 11.4      | Frontend Initialization (Angular) . . . . .              | 31        |
| <b>12</b> | <b>Lessons Learned</b>                                   | <b>31</b> |
| 12.1      | Successes and Positive Outcomes . . . . .                | 31        |
| 12.2      | Challenges Encountered . . . . .                         | 32        |
| 12.3      | Future Improvements and Alternative Approaches . . . . . | 32        |

# 1 Project Description

## 1.1 Problem Statement

In the current digital landscape, managing the daily operations of an auto repair shop continues to face major challenges due to fragmented administrative and economic processes. Coordinating daily activities involves complex data flows: scheduling clients based on physical ramp availability, dynamically allocating qualified mechanics, tracking parts inventory for each repair, accurate invoicing, and managing human resources (leave requests, time tracking, and workload).

Traditional or rudimentary systems (based on physical ledgers or isolated spreadsheet files) frequently generate operational errors, such as:

- **Overbooking:** Allocating the same time slot or the same mechanic for multiple vehicles simultaneously due to the lack of a real-time centralized state.
- **Poor Traceability:** Losing the repair history of a vehicle, which hinders accurate diagnostics during subsequent visits and affects the enforcement of parts and labor warranties.
- **Inefficient Staff Synchronization:** Approving mechanic leaves without automatically blocking their capacity to take on new cars during those intervals, creating workflow bottlenecks.
- **Web Security Vulnerabilities:** Exposing systems to brute-force attacks on authentication endpoints and risking data consistency when the same account is used concurrently from different browsers or through duplicated active tabs.

## 1.2 The Solution

The **AutoPulse** project is an enterprise-grade web platform specifically designed to centralize, automate, and secure the entire operational cycle of a modern auto repair shop. The system's architecture natively integrates operational and human resource management modules, offering a dedicated and optimized experience for four distinct user types (actors), managed through rigorous Role-Based Access Control (RBAC) mechanisms:

1. **The Client:** Benefits from a personalized portal to register and manage their vehicles, initiate online appointments by selecting available dates, and view the complete history of their past repairs (replaced parts, labor costs) in real-time.
2. **The Mechanic:** Has a simplified interface to view the daily flow of assigned tasks. They can update the repair status (from pending to completed), add detailed repair records and used parts, and request days off directly through the platform.
3. **The Manager:** Holds administrative and financial control over the shop. They oversee shop performance through aggregated metrics (revenue generated, volume of processed cars), approve or reject staff leave requests, and manage the issuance of invoices associated with completed repairs.

4. **The Administrator (Admin):** Responsible for the technical maintenance of the platform, global user management, and configuring system roles and permissions.

### 1.3 What Makes It Unique

Unlike generic commercial solutions, **AutoPulse** stands out by implementing advanced mechanisms for session state consistency and software infrastructure security:

- **Real-Time Tab Synchronization and Isolation:** The platform solves a critical issue of modern web applications by utilizing a custom `TabSyncService` based on the HTML5 `BroadcastChannel` API. When a user attempts to duplicate an active tab in the browser, the system instantly detects the duplicate instance, invalidates the session on the old tab, and redirects the user to the authentication screen, preventing application state alteration or Session Hijacking attacks.
- **Hybrid Concurrent Session Control (Token Versioning):** Although it uses stateless authentication based on JSON Web Tokens (JWT) for high scalability, the system introduces a hybrid tracking mechanism for the token version (`tokenVersion`) saved in the database within the `User` entity. Any new login attempt (from another device or browser) increments this version, causing the backend security filter (`JwtAuthFilter`) to instantly reject requests from older sessions, guaranteeing a strict single active session per account.
- **HR-Operational Correlation Algorithm:** The scheduling engine automatically checks for overlaps with the approved leave status of mechanics. A mechanic on confirmed leave is automatically excluded from the task distribution algorithm, completely eliminating human scheduling errors.
- **Infrastructure Protection via Rate Limiting:** The integration of a custom `RateLimitFilter` at the Spring Security pipeline level protects the API against Brute Force attacks on the Login and Register forms, blocking abusive IP addresses through a controlled token bucket mechanism.

## 2 Team & Contributions

### 2.1 Team Composition and Roles Assignment

The development of the **AutoPulse** platform was carried out by a multidisciplinary team of four members. To ensure maximum efficiency and maintain high software engineering standards, responsibilities were strictly segregated into specialized operational domains: Project Planning & Requirements Management, UI/UX Design, and core Software Engineering (Backend and Frontend). This strategic division of labor minimized development bottlenecks and enabled a highly parallelized workflow.

### 2.2 Detailed Contributions

Below is the detailed breakdown of the specific roles, core responsibilities, and key architectural components engineered by each team member during the lifecycle of the project:

### **Fret Dario – Project Planning & Requirements Lead**

- Conducted the initial system requirement analysis and established the functional project scope.
- Authored formal use case specifications, refined user stories, and mapped out functional requirements.
- Designed the comprehensive technical documentation blueprint and structured the agile project backlog.
- Coordinated timeline milestones, sprint objectives, and verified final architectural compliance.

### **Gîrz Ionuț – UI/UX & Visual Design Lead**

- Created responsive wireframes and high-fidelity interactive mockups for all user portals.
- Developed the global visual identity, layout typography, and consistent custom SCSS styling.
- Designed the intuitive dashboard structures, analytical chart layouts, and user-centric input forms.
- Ensured accessibility standards and visual uniformity across the Client, Mechanic, Manager, and Admin interfaces.

### **Cozi David – Lead Backend Developer (Software Engineering)**

- Designed the relational database schema utilizing JPA Inheritance (JOINED strategy) for the user hierarchy.
- Configured the Spring Security filter chain and implemented stateless JWT token generation.
- Engineered the concurrent session control mechanism via database-backed `tokenVersion` tracking to invalidate multi-device access.
- Implemented REST controllers, transactional business services, and the infrastructure-protecting `RateLimitFilter`.

### **Dascălu Raul – Lead Frontend Developer (Software Engineering)**

- Set up the Angular 17 enterprise workspace using standalone components and optimized lazy-loaded routing.
- Programmed the real-time `TabSyncService` utilizing the `HTML5 BroadcastChannel` API to achieve multi-tab isolation.
- Implemented the global `HTTP AuthInterceptor` for automated JWT injection and seamless 401 unauthorized error interception.
- Connected frontend component states with backend REST endpoints using reactive asynchronous RxJS streams.

## 2.3 Workflow and Collaboration Methodology

To maximize productivity and guarantee the integrity of the codebase, the team adhered to strict software development paradigms:

- **Version Control and Integration:** Git was utilized via GitHub, following a feature-branch workflow. Main branch stability was preserved by ensuring all local compilations and cross-tab security tests passed successfully prior to merging.
- **Decoupled Execution:** The strict separation of the Angular frontend from the Spring Boot backend allowed the two software engineers to work concurrently against a pre-negotiated REST API contract.
- **Design-Driven Development:** The frontend components were programmed directly from the high-fidelity specifications prepared during the design phase, streamlining the development process.

## 3 Actors & User Stories

### 3.1 Actors Definition

To ensure a rigorous mapping of functional requirements to the granular structure of the application's portals, the **AutoPulse** system defines four distinct user classes. Each actor interacts with the system through a dedicated dashboard secured by Role-Based Access Control (RBAC) mechanisms:

- **The Client:** The end-user of the platform and owner of one or more vehicles. They interact with the system asynchronously to schedule technical interventions, monitor the current status of repairs, and review vehicle service history by dynamically filtering their appointment lists. From a security perspective, their actions are strictly limited to resources under their own possession.
- **The Mechanic:** The operational technical staff responsible for the physical execution of repairs on the ramps. The mechanic does not have global visibility over the database; they exclusively access their own workflow of tasks (active appointments assigned by the system), update the technical status of the car, and log the details of the repairs. They also initiate human resource administrative flows (leave requests).
- **The Manager:** The administrative user with extended supervisory rights at the shop level. They have access to aggregated financial metrics, control workload capacity planning by validating staff leaves, and issue primary fiscal documents (invoices) following the technical reports closed by mechanics.
- **The Admin:** The technical superuser responsible for the platform's infrastructural integrity. Their main role is to manage the lifecycle of user accounts, apply corrections to nomenclatures, and dynamically modify system roles according to the shop's internal organizational chart.

## 3.2 User Stories and Acceptance Criteria

The following details the fundamental User Stories, structured according to software engineering best practices, and aligned with the functionalities implemented in the frontend and backend modules:

### 1. Client Portal

#### 1. Private Car Fleet Registration and Management

- *User Story:* As a Client, I want to register my vehicles by inputting the make, model, manufacturing year, and license plate, so that I can quickly select them when initiating a booking.
- *Acceptance Criteria:* The system must validate the uniqueness of the license plate and save the entity in the `cars` table, linking it directly to the current user's ID. Incorrectly entered data must be editable via the interface.

#### 2. Online Scheduling of Technical Interventions

- *User Story:* As a Client, I want to select a vehicle from my garage and pick an available date from the calendar for an appointment, specifying a description of the issue, so that I avoid waiting times at the shop.
- *Acceptance Criteria:* The request sent to the `POST /api/appointments` endpoint must automatically check the total capacity of the shop for that day (preventing overbooking) and set the initial status to `PENDING`.

#### 3. Tracking Vehicle History via Appointment Filtering

- *User Story:* As a Client, I want to filter my list of appointments by a specific vehicle, so that I can easily track all past interventions, descriptions of work done, and status updates associated with that car over time.
- *Acceptance Criteria:* The interface must provide a filtering component within the appointments view. Selecting a specific vehicle must dynamically filter the list to display only the appointments mapped to that vehicle's ID.

### 2. Mechanic Workspace

#### 1. Viewing and Managing Allocated Tasks

- *User Story:* As a Mechanic, I want to have a clear list of the appointments assigned to me for the current day, so that I know which cars to take on the ramp.
- *Acceptance Criteria:* The system filters records in the `appointments` table based on the authenticated mechanic's ID and the current date, displaying the details entered by the client.

#### 2. Real-Time Workflow Updates

- *User Story:* As a Mechanic, I want to be able to modify the status of an appointment (from `ACCEPTED` to `IN_PROGRESS` and later to `COMPLETED`), adding a description of the executed repair, so that the manager can see the completed work and handle invoicing.



- *Acceptance Criteria:* Upon transitioning to the **COMPLETED** state, the system updates the appointment state in the database, locking the description and making it available for financial processing.

### 3. Leave Planning via Platform

- *User Story:* As a Mechanic, I want to submit a leave request specifying the start and end dates, so that my absence is officially registered.
- *Acceptance Criteria:* The request is saved in the **PENDING** state in the **mechanic\_leaves** table and triggers an implicit notification in the manager's dashboard.

## 3. Manager Dashboard

### 1. Workload Allocation and Mechanic Assignment

- *User Story:* As a Manager, I want to manually distribute incoming service tasks or reassign an existing appointment to a specific mechanic based on their current availability, so that the shop's operational capacity is balanced and bottlenecks are avoided.
- *Acceptance Criteria:* The system must provide a list of active mechanics alongside their current task load. When an assignment is confirmed, the system must update the respective appointment record with the selected mechanic's ID and immediately update both the manager's metrics and the mechanic's task queue.

### 2. Human Resources Capacity Control

- *User Story:* As a Manager, I want to view a list of all mechanic leave requests and be able to approve or reject them, so that I can ensure business continuity on the shop floor.
- *Acceptance Criteria:* Changing the status to **APPROVED** triggers a logic sequence in the system that excludes the mechanic from the automatic allocation algorithm for the selected time frame.

### 3. Invoicing and Financial Closure of Records

- *User Story:* As a Manager, I want to automatically generate a fiscal invoice based on the technical details logged upon appointment completion, so that I can collect payment for the services.
- *Acceptance Criteria:* The system calculates the total cost, generating a record in the **invoices** table linked directly to the corresponding appointment.

## 4. Admin Portal

### 1. Access Level and Security Management

- *User Story:* As an Admin, I want to be able to modify a user account's role (e.g., from Client to Mechanic or Mechanic to Manager) and deactivate inactive accounts, so that I can maintain the platform's organizational security.
- *Acceptance Criteria:* Modifying the role via `PUT /api/users/id/role` must update the linkage table corresponding to the JPA Inheritance strategy and force the increment of the `tokenVersion` to instantly disconnect the active sessions of that user from other devices.

## 4 UML Diagrams

### 4.1 Use Case Diagram

The Use Case Diagram establishes the structural boundary of the **AutoPulse** ecosystem, highlighting the functional workflows executed by each of the four distinct authenticated actors.



Figure 1: UML Use Case Diagram showing Actors, Boundaries, and Includes/Extends dependencies.

## 4.2 Class Diagram

The Class Diagram illustrates the backend domain model and the Object-Relational Mapping (ORM) entities mapped via Spring Data JPA. At the core of the AutoPulse data architecture is the **User** entity, which implements the JPA **JOINED** inheritance strategy to efficiently manage the four distinct system roles: **Client**, **Mechanic**, **Manager**, and **Admin**.

This architectural choice allows the system to share common authentication credentials and security states—most notably the **tokenVersion** attribute used for concurrent session invalidation—while maintaining strictly separated relational tables for role-specific properties (e.g., a Mechanic’s **specialization** and **hourlyRate**, or an Admin’s **accessLevel**).

The operational workflow is represented through explicit entity associations:

- **Ownership & Scheduling:** A **Client** owns multiple **Cars** (*One-to-Many*), and a **Car** can have multiple **Appointments**.
- **Execution & Tracking:** An **Appointment** is assigned to a **Mechanic** and strictly mapped (*One-to-One*) to a **RepairRecord**. The **RepairRecord** aggregates the labor costs and the specific **Parts** used during the service (*One-to-Many*).
- **Financials & HR:** The system generates an **Invoice** mapped directly to the repair record. Additionally, human resources are managed via the **MechanicLeave** entity, allowing mechanics to request time off, which directly impacts the assignment algorithms.

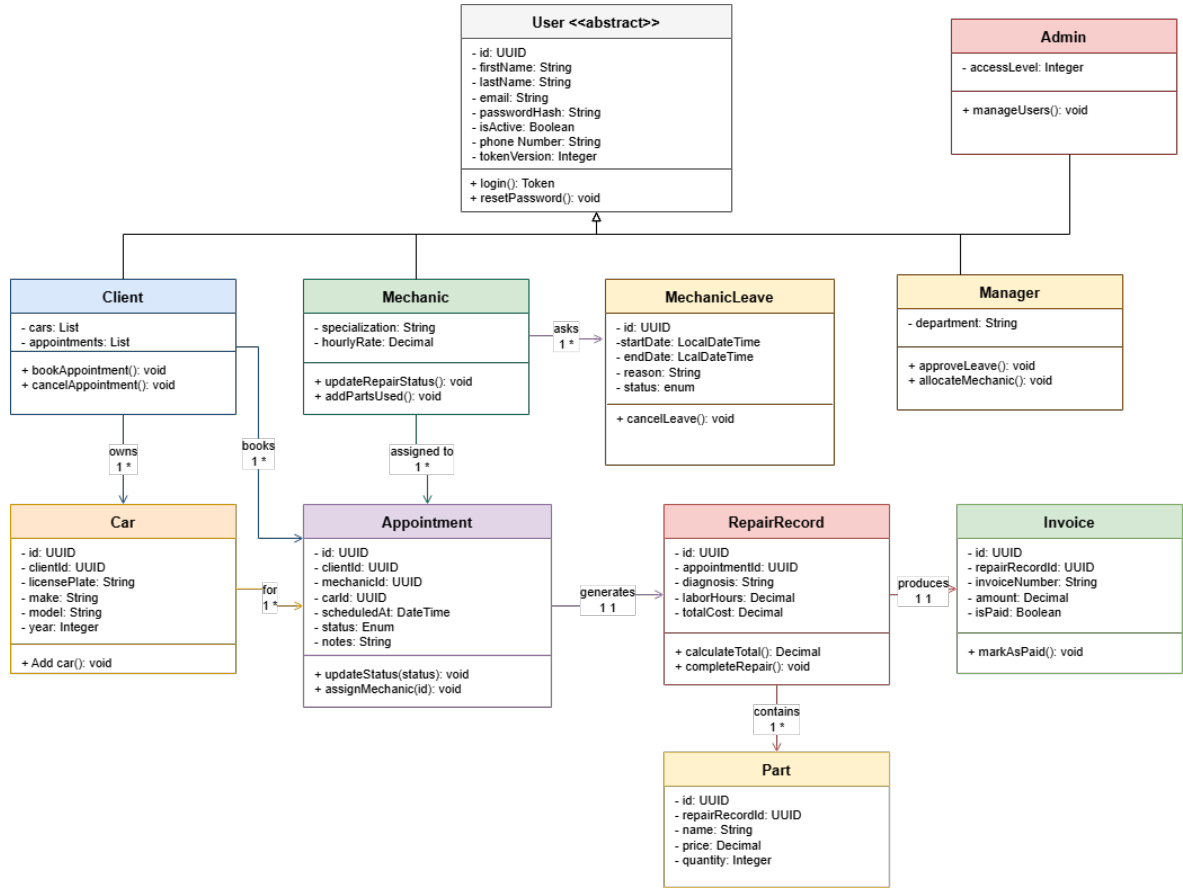


Figure 2: UML Class Diagram illustrating the Spring Boot domain models, JPA JOINED inheritance hierarchy, and cardinal entity relationships.

### 4.3 Sequence Diagram

The Sequence Diagram below illustrates a core operational flow within the **AutoPulse** platform. It details the step-by-step interactions between the actors, the user interface, the controller layer, and the database during a standard request lifecycle.

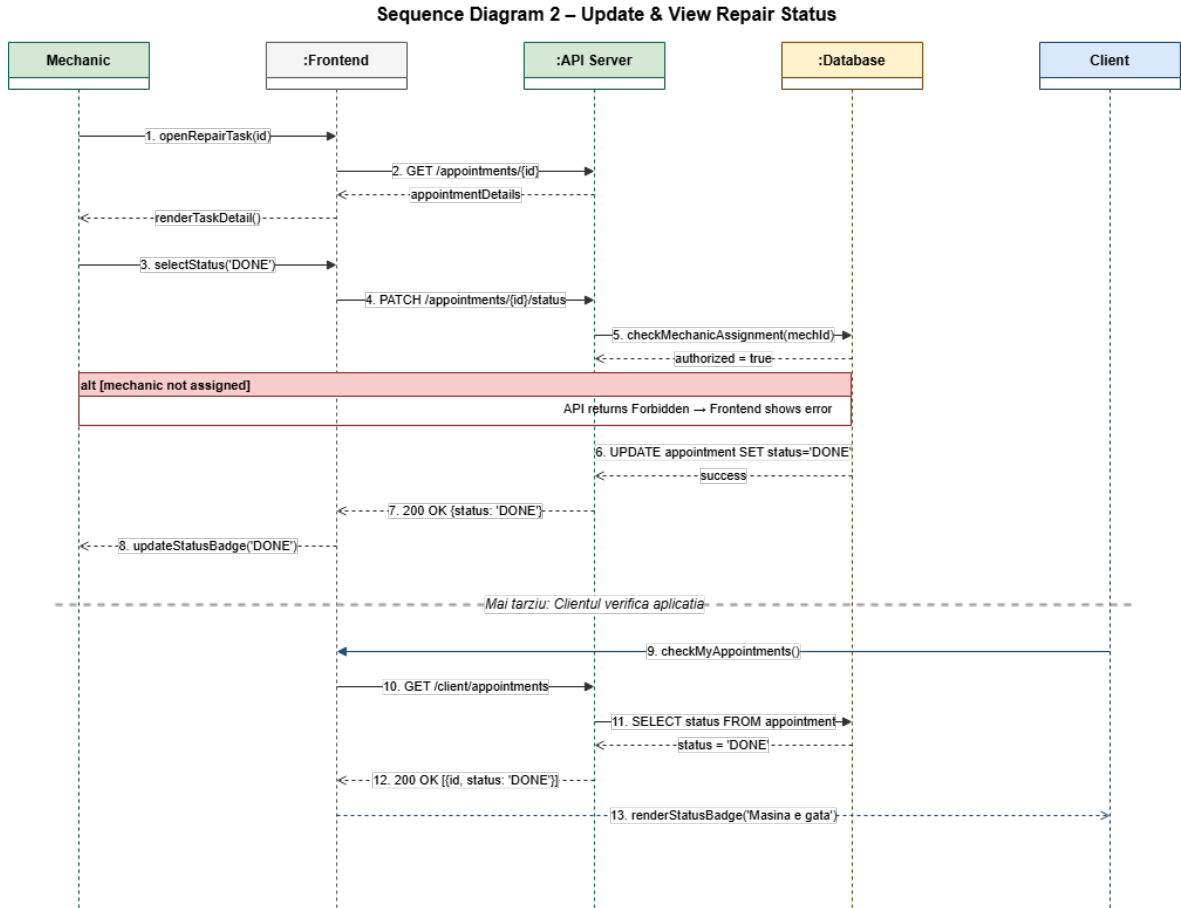


Figure 3: UML Sequence Diagram representing an example flow in the AutoPulse system.

## 5 Architecture

### 5.1 System Architecture Overview

The **AutoPulse** platform implements a decoupled, client-server architectural paradigm designed to optimize system security, high availability, and strict state consistency. The application separation boundaries isolate the user interface layer from the transactional application server. The system is split into two primary core components: a modern Single Page Application (SPA) frontend engineered with Angular, and an enterprise-grade monolithic REST API backend powered by Spring Boot.

Communication between the web browser tier and the underlying application server occurs asynchronously over secure HTTP channels using standardized RESTful design patterns. All operational data payloads are serialized and exchanged strictly in JSON (JavaScript Object Notation) format.

### 5.2 Technology Stack Breakdown

The application infrastructure integrates a strictly selected set of enterprise technologies to enforce clean separation of concerns and optimal runtime performance:

- **Frontend Layer:** Built using **Angular 21** leveraging modern standalone component architectures. Asynchronous data streams, reactive HTTP calls, and global state events are fully managed via **RxJS** observables. Dynamic layouts and responsive user interfaces are styled natively using **SCSS** and **HTML5**.
- **Backend Layer:** Powered by **Java 25** and the **Spring Boot 4+** framework ecosystem. Core system operations, cross-origin resource sharing (CORS), role validation, and state protection filters are executed entirely via the **Spring Security** pipeline.
- **Persistence Layer:** Employs a relational database management system (**PostgreSQL**). Database schema mapping, transactional boundaries, and object-relational mapping are fully abstracted using **Spring Data JPA** and Hibernate, enforcing strict foreign key constraints and implementing the table-per-subclass inheritance mechanism via the **JOINED** strategy.

### 5.3 System Architecture Diagram

The structural configuration of these software components, alongside the filtering layers governing web traffic and authentication, is visualized in the architectural diagram below:

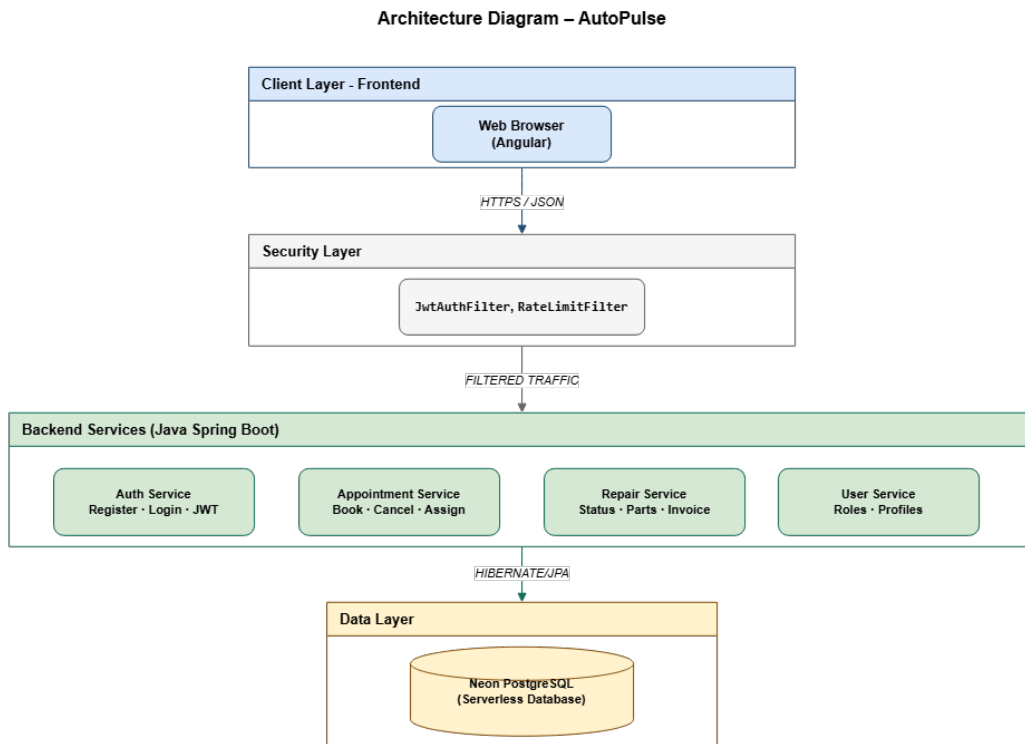


Figure 4: AutoPulse System Architecture Diagram showcasing the Angular frontend subsystems, the Spring Boot security pipeline, and the relational database persistence tier.

## 5.4 Key Architectural Subsystems

To maximize security and mitigate data inconsistency across clients, several specialized architectural subsystems were engineered into the application lifecycle:

1. **Client-Side Session Interception (AuthInterceptor):** An Angular HTTP interceptor that monitors outgoing network requests. It automatically extracts the stateless JSON Web Token (JWT) from secure browser storage and seamlessly injects it as a Bearer token into the Authorization headers of every secured REST endpoint call.
2. **Multi-Tab Session Isolation (TabSyncService):** To prevent concurrent state contamination and avoid synchronization discrepancies across multiple browser windows, this service leverages the HTML5 `BroadcastChannel` API to monitor real-time tab state duplicates and guarantee session safety.
3. **Infrastructural Rate Limiting (RateLimitFilter):** Positioned at the absolute entry point of the Spring Security filter chain, this filter implements a custom Token Bucket algorithm. It analyzes incoming client IP addresses to restrict rapid request bursts, successfully protecting public forms (Login, Register) from automated brute-force attacks.
4. **Hybrid Concurrency Control (JwtAuthFilter):** Although the application uses stateless JWTs for high horizontal scalability, the backend coordinates a hybrid database-backed validation rule. Every request intercepts the cryptographic token, validates its signature, and compares the session data with a database-stored `tokenVersion` integer. When a user logs in from a new location, the version increments, immediately invalidating old tokens across all other active devices.

## 6 Database Schema

The relational database schema is designed to ensure strict data consistency, eliminate structural redundancy, and facilitate optimized transactional processing for the AutoPulse platform.

### 6.1 Entity-Relationship Diagram (ERD)

The complete structural layout of the database, highlighting the individual tables, column data types, primary keys, and foreign key boundaries, is illustrated in the diagram below:

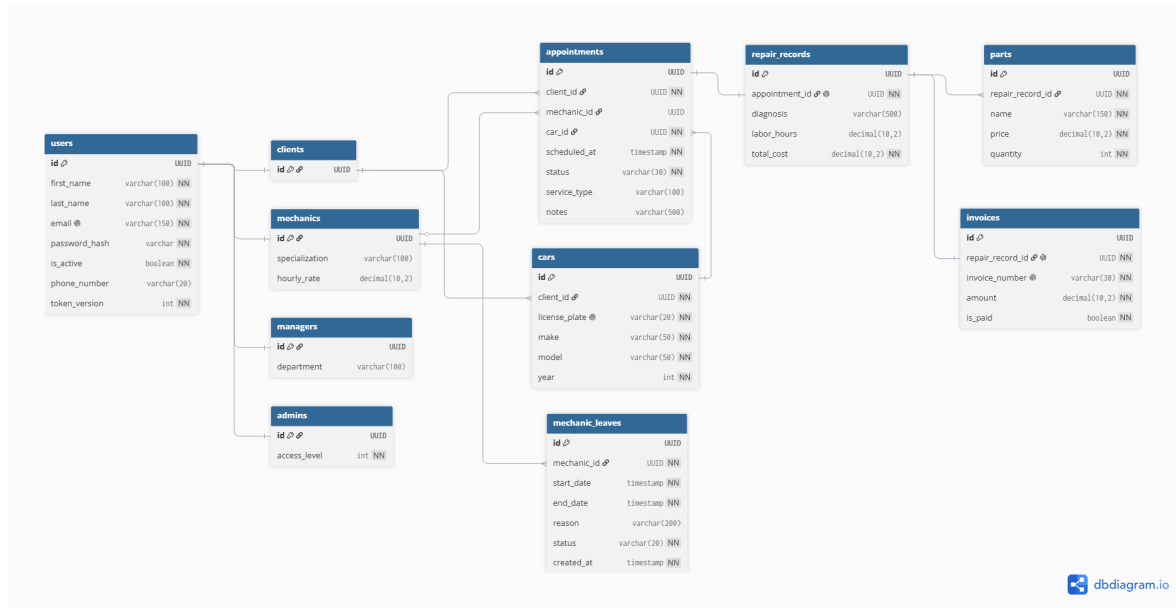


Figure 5: AutoPulse Entity-Relationship Diagram (ERD) generated via dbdiagram.io.

## 6.2 Schema Description and Relational Mapping

The database persistence layer leverages a table-per-subclass strategy (*JOINED* inheritance) combined with strong referential constraints to map business operations accurately:

### 6.2.1 User Inheritance Hierarchy

- **users Table:** Acts as the foundational data store containing baseline credentials and state data shared across all account tiers, including authentication properties (`email`, `password_hash`), contact info (`phone_number`), administrative status (`is_active`), and the security-critical `token_version` parameter used for active session enforcement.
- **clients, mechanics, managers, and admins Tables:** Represent role-specific extensions mapping back to the primary `users` table via a shared identifier acting simultaneously as a primary and foreign key constraint. The `mechanics` table stores specialized attributes such as `specialization` and `hourly_rate`, the `managers` table maintains administrative scope with `department`, and the `admins` table contains permission flags via `access_level`.

### 6.2.2 Core Operational Entities

- **cars Table:** Maps a client's private automotive fleet using a strict *Many-to-One* foreign key link (`client_id`) pointing to the `clients` table, enforcing unique identifiers for vehicle assets through a distinct constraint on `license_plate`.
- **appointments Table:** Functions as a centralized operational hub that dynamically binds a scheduling event to three target matrices: the issuing owner (`client_id`), the tracked asset (`car_id`), and the optionally designated technician (`mechanic_id`).



It logs scheduling parameters via `scheduled_at` and manages states through a bounded `status` enumeration string.

### 6.2.3 Technical Maintenance and Financial Ledger Entries

- **mechanic\_leaves Table:** Handles human resource scheduling availability by creating an explicit timeline ledger mapped to a technician (`mechanic_id`) using a defined boundaries array (`start_date` and `end_date`).
- **repair\_records Table:** Forges a tight *One-to-One* relational bond with an active `appointment_id` upon technical service closure. This table isolates analytical logging parameters (`diagnosis`), quantifies diagnostic labor variables (`labor_hours`), and computes financial aggregations (`total_cost`).
- **parts Table:** Tracks inventory dissipation using a *Many-to-One* dependency array connected to a specific service catalog entry via `repair_record_id`.
- **invoices Table:** Finalizes the transactional boundary by implementing a strict *One-to-One* dependency on a closed `repair_record_id`, storing a validated billing sequence (`invoice_number`), cost metrics (`amount`), and collection states (`is_paid`).

## 7 API Documentation

### 7.1 API Architecture and Protocol Specifications

The **AutoPulse** platform exposes a stateless, enterprise-grade RESTful API architecture over secure HTTP layers. Communication across the decoupled system boundaries uses standard HTTP verbs (`GET`, `POST`, `PUT`, `DELETE`) bound to distinct operational controller matrices. Data serialization and exchange are executed exclusively using JavaScript Object Notation (JSON) payloads.

#### 7.1.1 Global HTTP Status Code Matrix

Endpoints adhere strictly to semantic REST guidelines to propagate operational states and execution boundaries back to the Angular client application:

- **200 OK:** Request successfully executed; transactional payload encapsulated in the response body.
- **201 Created:** Resource creation operation successful (e.g., car registration, appointment booking).
- **400 Bad Request:** Validation failures on input schemas, malformed JSON objects, or logical constraint violations.
- **401 Unauthorized:** The session JWT token is completely absent, cryptographically malformed, or has expired.
- **403 Forbidden:** The authenticated user's assigned role (RBAC) does not possess sufficient privileges to execute the action.
- **429 Too Many Requests:** The infrastructure-protecting `RateLimitFilter` bucket has been depleted due to rapid successive operations from the calling IP address.

### 7.1.2 Security and Authentication Headers

Except for endpoints explicitly categorized as PUBLIC, all secured endpoints require the client to supply a valid JWT token via the HTTP request headers wrapper:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Listing 1: Mandatory Authorization Header Specification

## 7.2 Comprehensive Endpoint Map

The complete operational REST surface area of the backend system is aggregated in the design matrix below:

| Method                                                     | URI Endpoint                         | Role Scope   | Transactional Purpose                              |
|------------------------------------------------------------|--------------------------------------|--------------|----------------------------------------------------|
| Authentication & Session Infrastructure Module (/api/auth) |                                      |              |                                                    |
| POST                                                       | /api/auth/register                   | PUBLIC       | Registers a new Client account tier                |
| POST                                                       | /api/auth/login                      | PUBLIC       | Authenticates user and returns JWT                 |
| PATCH                                                      | /api/profile/password                | ALL AUTH     | Modifies active user credentials safely            |
| PATCH                                                      | /api/profile                         | ALL AUTH     | Modifies profile contact parameters                |
| Automotive Asset Fleet Module (/api/cars/my)               |                                      |              |                                                    |
| GET                                                        | /api/cars/my                         | CLIENT       | Fetches private garage car objects                 |
| POST                                                       | /api/cars/my                         | CLIENT       | Appends a new car entity to the fleet              |
| Service Scheduling Module (/api/appointments)              |                                      |              |                                                    |
| GET                                                        | /api/appointments                    | FILTERED     | Fetches scoped appointments array                  |
| POST                                                       | /api/appointments                    | CLIENT       | Books an appointment event slot                    |
| PUT                                                        | /api/appointments/{id}/status        | TECH/MANAGER | Mutates operational appointment lifecycle          |
| PUT                                                        | /api/appointments/{id}/assign        | MANAGER      | Distributes task to a targeted mechanic            |
| Technical Logs & Diagnostics Module (/api/repair-records)  |                                      |              |                                                    |
| POST                                                       | /api/repair-records                  | MECHANIC     | Closes service task with an active repair log      |
| GET                                                        | /api/appointments/{id}/repair-record | ALL AUTH     | Fetches diagnostic ledger data                     |
| Human Resource Scheduling Module (/api/leaves)             |                                      |              |                                                    |
| POST                                                       | /api/leaves                          | MECHANIC     | Submits a timeline leave allocation entry          |
| GET                                                        | /api/leaves                          | MECH/MANAGER | Reviews active or pending timeline states          |
| PUT                                                        | /api/leaves/{id}/status              | MANAGER      | Evaluates and changes leave permission states      |
| Financial Ledger & Invoicing Module (/api/invoices)        |                                      |              |                                                    |
| GET                                                        | /api/invoices                        | MGR/CLIENT   | Fetches historical and outstanding invoices        |
| Analytics & System Metrics Module (/api/stats)             |                                      |              |                                                    |
| GET                                                        | /api/stats/dashboard                 | MANAGER      | Aggregates operational performance indices         |
| PUT                                                        | /api/users/{id}/role                 | ADMIN        | Alters systemic roles and increments token version |

Table 1: Comprehensive REST API Endpoint Inventory Mapping Matrix

## 7.3 Granular Endpoint Specifications and Payload Examples

### 7.3.1 Authentication & Profile Module

#### 1. User Registration Schema

- **Endpoint:** POST /api/auth/register
- **Access Modifier:** PUBLIC
- **Description:** Creates a base user and maps it to a Client database relationship.

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "password": "SecurePassword123!",
}
```

```
"phoneNumber": "+40722111222"
}
```

Listing 2: Registration Request Body Sample

```
{
  "id": "e4a8b2c6-d1e3-4f5a-a6b7-c8d9e0f1a2b3",
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "role": "CLIENT",
  "isActive": true
}
```

Listing 3: Registration Response Body (201 Created) Sample

## 2. Authentication Login Handshake

- **Endpoint:** POST /api/auth/login
- **Access Modifier:** PUBLIC
- **Description:** Validates password hashes, triggers rate constraints, increments tokenVersion checking, and crafts a bearer cryptographic token.

```
{
  "email": "john.doe@example.com",
  "password": "SecurePassword123!"
}
```

Listing 4: Login Request Body Sample

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJqb2huLmRvZUBleGFtcGxlLmNvbSIsImklIjoiaZTRhOGIyYzYtZDFlMy00ZS5SignatureValidatedHere...",
  "id": "e4a8b2c6-d1e3-4f5a-a6b7-c8d9e0f1a2b3",
  "email": "john.doe@example.com",
  "role": "CLIENT",
  "firstName": "John",
  "lastName": "Doe"
}
```

Listing 5: Login Response Body (200 OK) Sample

## 3. Credentials Alteration Flow

- **Endpoint:** PATCH /api/profile/password
- **Access Modifier:** AUTHENTICATED (ANY ROLE)

```
{
  "oldPassword": "SecurePassword123!",
  "newPassword": "NewHighlySecurePassword987!"
}
```

Listing 6: Change Password Request Body Sample

### 7.3.2 Automotive Asset Fleet Module

#### 1. Garage Inventory Expansion

- **Endpoint:** POST /api/cars/my
- **Access Modifier:** CLIENT

```
{
  "licensePlate": "BH99XYZ",
  "make": "BMW",
  "model": "3 Series",
  "year": 2021
}
```

Listing 7: Car Creation Request Body Sample

```
{
  "id": "7b1c3d5e-7f9a-4b2c-8d1e-3f5a7b9c1d3e",
  "licensePlate": "BH99XYZ",
  "make": "BMW",
  "model": "3 Series",
  "year": 2021,
  "clientId": "e4a8b2c6-d1e3-4f5a-a6b7-c8d9e0f1a2b3"
}
```

Listing 8: Car Creation Response Body (201 Created) Sample

#### 2. Vehicle Fleet Evaluation

- **Endpoint:** GET /api/cars/my
- **Access Modifier:** CLIENT

```
[
  {
    "id": "7b1c3d5e-7f9a-4b2c-8d1e-3f5a7b9c1d3e",
    "licensePlate": "BH99XYZ",
    "make": "BMW",
    "model": "3 Series",
    "year": 2021
  }
]
```

Listing 9: Retrieve Registered Cars Response Body (200 OK) Sample

### 7.3.3 Service Scheduling Module

#### 1. Appointment Event Dispatch

- **Endpoint:** POST /api/appointments
- **Access Modifier:** CLIENT

```
{
  "carId": "7b1c3d5e-7f9a-4b2c-8d1e-3f5a7b9c1d3e",
  "scheduledAt": "2026-06-15T09:00:00",
  "serviceType": "Engine Oil & Filters Replacement",
  "notes": "Please check abnormal brake squeaking noise as well
. "
}
```

Listing 10: Appointment Booking Request Body Sample

```
{
  "id": "1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d",
  "carId": "7b1c3d5e-7f9a-4b2c-8d1e-3f5a7b9c1d3e",
  "carLicensePlate": "BH99XYZ",
  "carDetails": "BMW 3 Series (2021)",
  "scheduledAt": "2026-06-15T09:00:00",
  "status": "SCHEDULED",
  "serviceType": "Engine Oil & Filters Replacement",
  "notes": "Please check abnormal brake squeaking noise as well
. ",
  "mechanicId": null
}
```

Listing 11: Appointment Booking Response Body (201 Created) Sample

#### 2. Resource Workload Allocation and Technical Assignment

- **Endpoint:** PUT /api/appointments/{id}/assign
- **Access Modifier:** MANAGER
- **Description:** Allows a shop manager to allocate tasks or adjust operational balanced capacities by explicitly linking a technician.

```
{
  "mechanicId": "8f7e6d5c-4b3a-2l1k-0j9i-8h7g6f5e4d3c"
}
```

Listing 12: Workload Allocation Assignment Request Body Sample

```
{
  "id": "1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d",
  "status": "ASSIGNED",
  "mechanicId": "8f7e6d5c-4b3a-2l1k-0j9i-8h7g6f5e4d3c",
  "mechanicName": "Alexandru Popescu"
}
```

Listing 13: Workload Allocation Assignment Response Body (200 OK) Sample

### 3. Lifecycle State Mutation

- **Endpoint:** PUT /api/appointments/{id}/status
- **Access Modifier:** MECHANIC / MANAGER

```
{  
  "status": "IN_PROGRESS"  
}
```

Listing 14: Status Mutation Request Body Sample

```
{  
  "id": "1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d",  
  "status": "IN_PROGRESS",  
  "updatedAt": "2026-05-24T22:15:40"  
}
```

Listing 15: Status Mutation Response Body (200 OK) Sample

#### 7.3.4 Technical Maintenance Financial Ledger Module

##### 1. Repair Record Logging and Technical Closure

- **Endpoint:** POST /api/repair-records
- **Access Modifier:** MECHANIC
- **Description:** Submits a detailed diagnostic report, sets quantifiable labor metrics, lists materials expended, and triggers an automated downstream invoicing sequence.

```
{  
  "appointmentId": "1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d",  
  "diagnosis": "Replaced structural engine oil, cabin filters.  
    Inspected front brake calipers and replaced worn out front  
    brake pads.",  
  "laborHours": 2.5,  
  "parts": [  
    {  
      "name": "Synthetic Engine Oil 5W30 (5L)",  
      "price": 250.00,  
      "quantity": 1  
    },  
    {  
      "name": "Brembo Front Brake Pads Set",  
      "price": 420.50,  
      "quantity": 1  
    }  
  ]  
}
```

Listing 16: Repair Logging Request Body Sample

```
{
  "id": "9c8b7a6f-5e4d-3c2b-1a0f-9e8d7c6b5a4f",
  "appointmentId": "1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d",
  "diagnosis": "Replaced structural engine oil, cabin filters.
    Inspected front brake calipers and replaced worn out front
    brake pads.",
  "laborHours": 2.5,
  "laborCost": 250.00,
  "partsCost": 670.50,
  "totalCost": 920.50,
  "invoiceNumber": "INV-2026-00041"
}
```

Listing 17: Repair Logging Response Body (201 Created) Sample

### 7.3.5 Human Resource Management Module

#### 1. Leave Timeline Allocation Request

- **Endpoint:** POST /api/leaves
- **Access Modifier:** MECHANIC

```
{
  "startDate": "2026-07-01T00:00:00",
  "endDate": "2026-07-10T23:59:59",
  "reason": "Annual family summer vacation leave."
}
```

Listing 18: Leave Submission Request Body Sample

```
{
  "id": "c3b2a1f0-e9d8-c7b6-a5f4-e3d2c1b0a9f8",
  "mechanicId": "8f7e6d5c-4b3a-2l1k-0j9i-8h7g6f5e4d3c",
  "startDate": "2026-07-01T00:00:00",
  "endDate": "2026-07-10T23:59:59",
  "reason": "Annual family summer vacation leave.",
  "status": "PENDING",
  "createdAt": "2026-05-24T22:20:00"
}
```

Listing 19: Leave Submission Response Body (201 Created) Sample

#### 2. Leave Evaluation and Permission Mutation

- **Endpoint:** PUT /api/leaves/{id}/status
- **Access Modifier:** MANAGER

```
{
  "status": "APPROVED"
}
```

Listing 20: Leave Status Evaluation Request Body Sample

```
{
  "id": "c3b2a1f0-e9d8-c7b6-a5f4-e3d2c1b0a9f8",
  "status": "APPROVED",
  "evaluatedByManagerId": "3d2c1b0a-e9d8-c7b6-a5f4-c3b2a1f0e9d8"
}
```

Listing 21: Leave Status Evaluation Response Body (200 OK) Sample

### 7.3.6 System Metrics Administration Module

#### 1. Role Modification and Core Token Invalidation

- **Endpoint:** PUT /api/users/{id}/role
- **Access Modifier:** ADMIN
- **Description:** Performs table migrations corresponding to the inheritance mapping strategy and triggers an atomic database-backed increment on the targeted user's `tokenVersion`. This instantly blocks all external distributed browser tab session instances.

```
{
  "newRole": "MANAGER",
  "department": "Executive Technical Operations"
}
```

Listing 22: Administrative Role Modification Request Body Sample

```
{
  "userId": "8f7e6d5c-4b3a-2l1k-0j9i-8h7g6f5e4d3c",
  "previousRole": "MECHANIC",
  "updatedRole": "MANAGER",
  "tokenVersion": 1,
  "sessionStatus": "ALL_ACTIVE_TOKENS_INVALIDATED"
}
```

Listing 23: Administrative Role Modification Response Body (200 OK) Sample

## 8 Application Screenshots

This chapter provides a visual walkthrough of the **AutoPulse** platform, demonstrating the implemented User Interface (UI) and User Experience (UX). The frontend is engineered using Angular 21, ensuring a responsive, accessible, and intuitive design across all role-based dashboards.



## 8.1 Public Interface and Authentication

The entry point of the application enforces strict access control. The interfaces below illustrate the secure login gateway and the client registration form, both protected by the underlying rate-limiting infrastructure.

The figure displays two screenshots of the AutoPulse application's authentication interface, set against a dark background.

The top screenshot shows the "Sign in to your account" form. It features the AutoPulse logo at the top, followed by the title "Sign in to your account" and the subtitle "Manage your car service appointments". The form includes input fields for "EMAIL ADDRESS" (containing "cozidavi9@gmail.com") and "PASSWORD" (masked with dots). A red error message "Invalid credentials" is displayed below the password field. A blue "Sign in -->" button is at the bottom, with a link "Don't have an account? Register" below it.

The bottom screenshot shows the "Create your account" form. It features the AutoPulse logo at the top, followed by the title "Create your account" and the subtitle "Join AutoPulse today". The form includes input fields for "FIRST NAME" (containing "Marian"), "LAST NAME" (containing "Vasile"), "EMAIL ADDRESS" (containing "alex@example.com"), "PHONE NUMBER" (containing "0767380745"), and "PASSWORD" (masked with dots). A red error message "Please enter a valid email address." is displayed below the email field. A blue "Create account -->" button is at the bottom, with a link "Already have an account? Sign in" below it.

Figure 6: The Authentication module UI, featuring reactive form validation and dynamic error handling.

## 8.2 Client Workspace

The Client Dashboard allows users to manage their automotive fleet and schedule service interventions. The interface is designed to provide immediate visibility over pending and historical appointments.

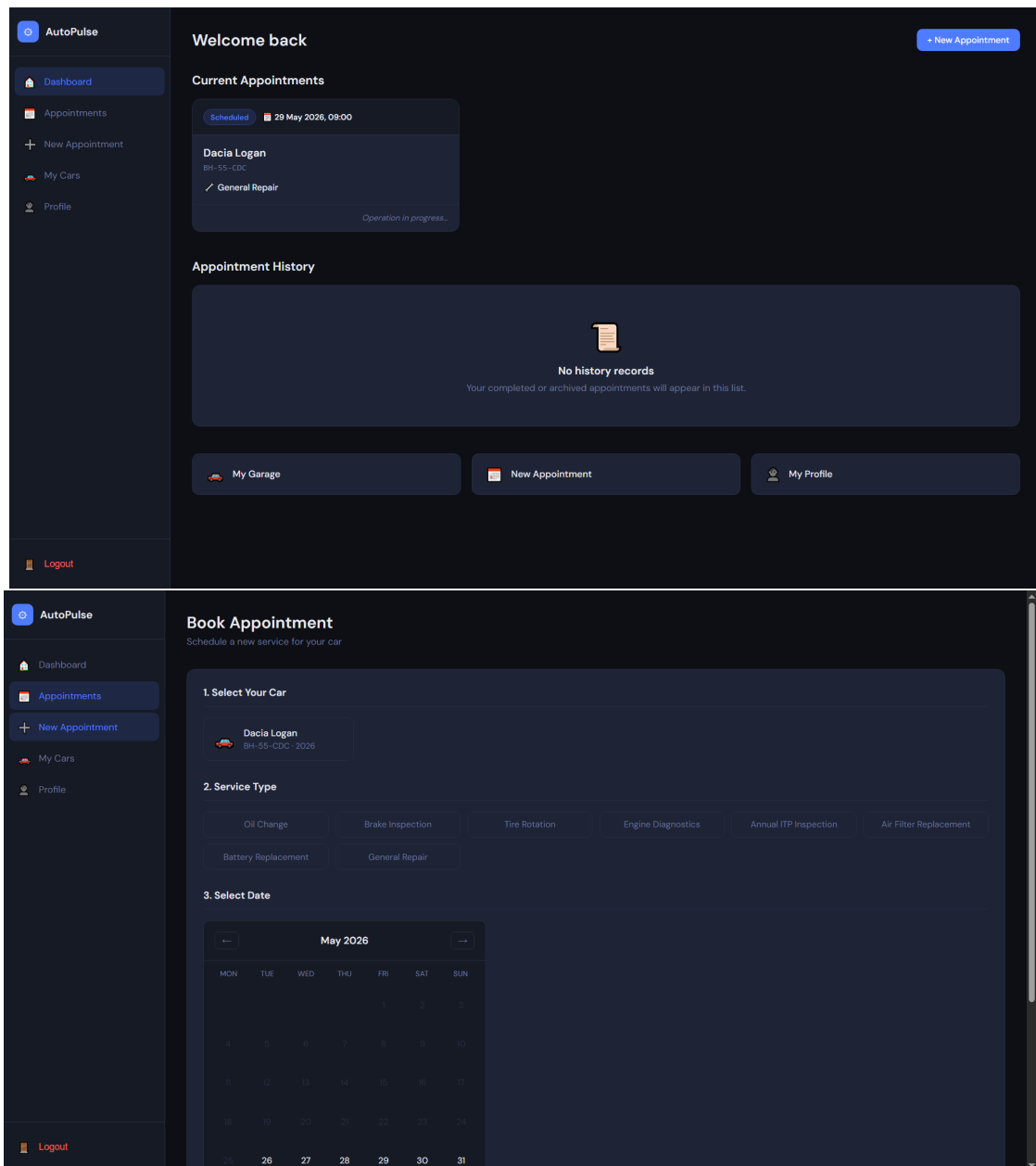


Figure 7: The Client Dashboard displaying the made appointment and the interactive appointment booking modal.

## 8.3 Mechanic Task Management

The Mechanic interface focuses on operational efficiency, allowing technicians to view assigned workloads.

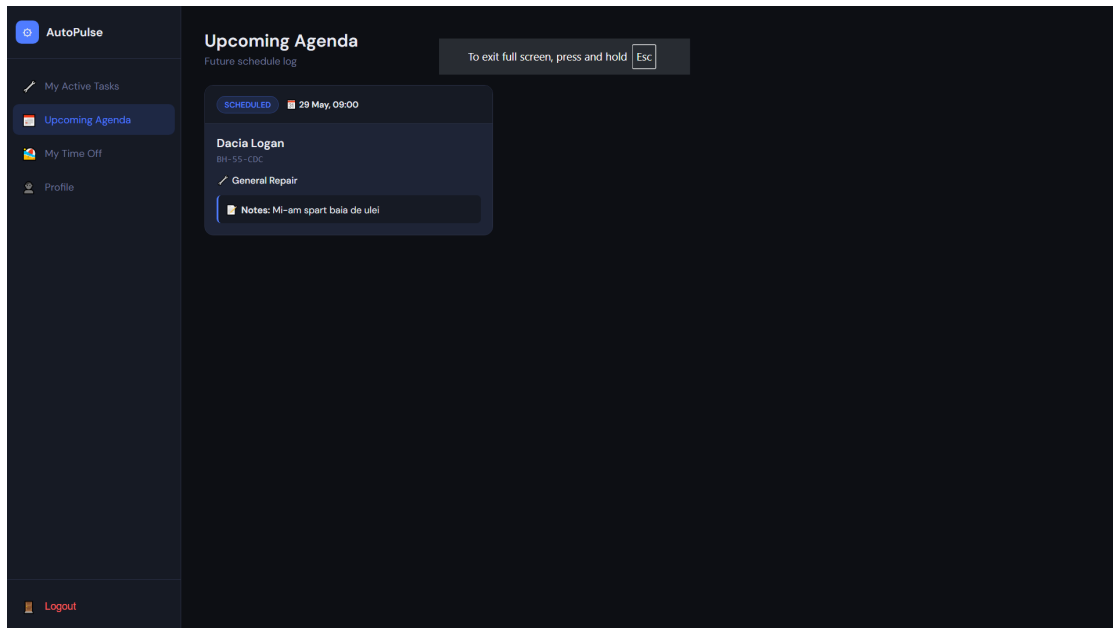


Figure 8: The Mechanic Workspace showing upcoming tasks

## 8.4 Managerial Dashboard

The Managerial view provides comprehensive oversight of the shop floor, including dynamic workload allocation to available mechanics and financial invoice generation.

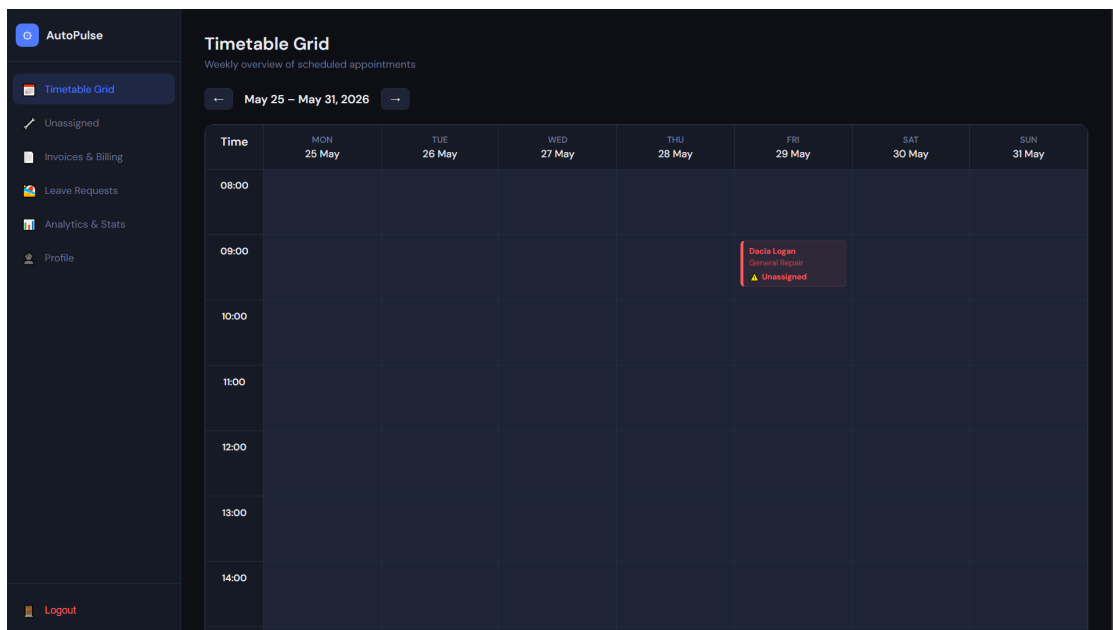


Figure 9: The Manager Dashboard illustrating the workload allocation grid and mechanic assignment workflow, alongside the admin dashboard illustrating user role mutations

## 8.5 Admin Dashboard

The Admin view allows for system-wide user role mutations.

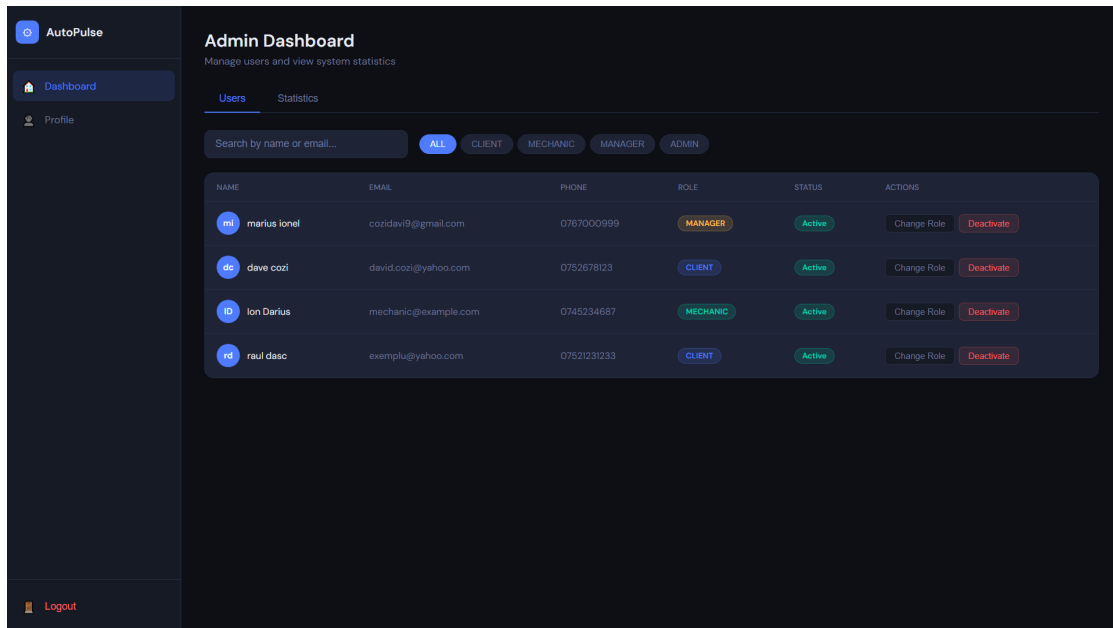


Figure 10: The admin dashboard illustrating user role mutations and account manipulation

## 9 Testing

### 9.1 Testing Strategy

Testing was performed manually through end-to-end usage scenarios, verifying the key functionalities and the role-based access control of the application.

## 9.2 Tested Scenarios

| #                                          | Scenario                       | Input / Condition                                                  | Expected Result                                               |
|--------------------------------------------|--------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------|
| <b>Authentication &amp; Access Control</b> |                                |                                                                    |                                                               |
| T1                                         | New user registration          | Valid email, password $\geq 8$ characters                          | Account created, redirect to dashboard page                   |
| T2                                         | Login with invalid credentials | Correct email, wrong password                                      | Error toast: "Invalid credentials!"                           |
| T3                                         | Role-based routing guard       | Login as CLIENT, attempt to access <code>/manager-dashboard</code> | Access denied, redirect to <code>/login</code> or home        |
| <b>Client Workflows</b>                    |                                |                                                                    |                                                               |
| T4                                         | Register new vehicle           | Complete make, model, year, and license plate                      | Vehicle appears immediately in the Client's garage list       |
| T5                                         | Book service appointment       | Select registered car, valid future date, and add notes            | Appointment saved in DB, status set to SCHEDULED              |
| <b>Mechanic Workflows</b>                  |                                |                                                                    |                                                               |
| T6                                         | Update appointment status      | Click "Start Work" on an assigned task                             | Status changes to IN_PROGRESS, UI updates accordingly         |
| T7                                         | Submit repair record           | Fill diagnosis, labor hours, and used parts                        | Record saved, fiscal Invoice automatically generated          |
| T8                                         | Submit leave request           | Select valid future start date and end date                        | Leave entry created, status set to PENDING                    |
| <b>Managerial &amp; Admin Workflows</b>    |                                |                                                                    |                                                               |
| T9                                         | Workload allocation            | Manager selects mechanic for a pending appointment                 | Mechanic assigned, appointment status updated                 |
| T10                                        | Leave overlap prevention       | Attempt to assign a mechanic currently on approved leave           | Mechanic is excluded from the available dropdown list         |
| T11                                        | Approve mechanic leave         | Manager clicks "Approve" on a pending leave request                | Status changes to APPROVED, HR capacity metrics updated       |
| T12                                        | Change user role (Admin)       | Admin changes a User's role to MANAGER                             | Role updated in DB, target user's active sessions invalidated |

Table 2: Manual End-to-End Testing Scenarios

## 9.3 Results

All 12 scenarios passed successfully. No critical functional errors or security bypasses were identified during the manual testing phase.

Minor observations:

- Some backend validation errors currently display a generic "Bad Request" toast on the frontend instead of mapping the specific field errors. This is a candidate for future UI refactoring.

## 10 Security Measures

The **AutoPulse** platform implements a multi-layered security architecture designed to protect sensitive user data, prevent unauthorized access, and mitigate common web vulnerabilities. Both the Spring Boot backend and the Angular frontend enforce strict security protocols.

### 10.1 Authentication and Authorization

- **JSON Web Tokens (JWT):** The system uses stateless JWTs for secure authentication. Upon successful login, the backend issues a cryptographically signed token containing the user's identity and role claims, which is validated on every subsequent request.
- **Role-Based Access Control (RBAC):** The API surface area is strictly partitioned using Spring Security. Endpoints are protected based on the `JOINED` table inheritance hierarchy, ensuring that specific transactional operations are restricted exclusively to their designated roles (`CLIENT`, `MECHANIC`, `MANAGER`, `ADMIN`).
- **Password Cryptography:** Raw passwords are never stored in the database. The system utilizes strong cryptographic hashing algorithms (e.g., BCrypt) to map user passwords into the `password_hash` database column, protecting credentials against database exposure.

### 10.2 Concurrent Session and State Security

- **Global Session Invalidation (`tokenVersion`):** To prevent stale or hijacked sessions, the backend implements a database-backed `tokenVersion` integer. Administrative actions (e.g., role modification) or user-driven security events (e.g., password change) instantly increment this value. The `JwtAuthFilter` intercepts all requests and compares the JWT's version payload with the database, instantly rejecting outdated tokens.
- **Multi-Tab State Isolation (`TabSyncService`):** The Angular frontend employs the HTML5 `BroadcastChannel` API to synchronize authentication states across multiple browser tabs. If a user logs out or is forcefully disconnected in one tab, all other active tabs are immediately synchronized and redirected to the public gateway, preventing data leaks.

### 10.3 Network and API Protection

- **Infrastructural Rate Limiting (`RateLimitFilter`):** To prevent automated brute-force attacks and volumetric API abuse (DDoS), a custom Token Bucket algorithm is integrated at the very top of the Spring Security filter chain. It actively monitors client IP addresses and throttles rapid successive requests, particularly on public endpoints like `/api/auth/login`.
- **Secure Payload Interception (`AuthInterceptor`):** The frontend HTTP client is configured with an interceptor that automatically attaches the Bearer JWT to the `Authorization` header. This ensures consistent authentication propagation without exposing token handling logic directly within the UI components.

# 11 Setup Instructions

This section provides a comprehensive step-by-step guide to configuring, building, and launching the **AutoPulse** platform from scratch in a local development environment.

## 11.1 Prerequisites

Ensure the following core technologies are installed and properly configured on the host machine prior to deployment:

- **Java Development Kit (JDK) 25:** Required for compiling the Spring Boot backend.
- **Node.js and npm:** Required for managing frontend dependencies.
- **Angular CLI 21:** Global installation is recommended (`npm install -g @angular/cli`).
- **Relational Database:** A running instance of PostgreSQL listening on its default port.

## 11.2 Environment and Database Configuration

1. Initialize the local database server and create a new schema named `autopulse_db`.
2. Navigate to the backend configuration directory: `backend/src/main/resources/`.
3. Open the `application.properties` (or `env.properties`, provided it is mapped via `@PropertySource`) file and set the database credentials alongside the JWT parameters:

```
# Database Configuration
spring.datasource.url=jdbc:postgresql://localhost:5432/
    autopulse_db
spring.datasource.username=your_db_username
spring.datasource.password=your_db_password
spring.jpa.hibernate.ddl-auto=update

# JWT Security Configuration
jwt.secret=YourSuperSecretKeyHere...
jwt.expiration=86400000
```

Listing 24: Local Environment Configuration

## 11.3 Backend Initialization (Spring Boot)

1. Open the backend repository in the preferred Integrated Development Environment (IDE) equipped with Spring tools (e.g., VS Code, Eclipse).
2. Open a terminal inside the IDE at the root of the backend project and execute the Maven Wrapper command to resolve dependencies cleanly (bypassing global path constraints):

- On Windows: `.\mvnw.cmd clean install`
  - On macOS/Linux: `./mvnw clean install`
3. Locate the **Spring Boot Dashboard** extension within the IDE interface.
  4. Launch the application server by finding the project in the dashboard and clicking the corresponding **Start/Run** button.
  5. The Spring Boot REST API will initialize the internal Tomcat server and listen for incoming HTTP connections on `http://localhost:8080`.

## 11.4 Frontend Initialization (Angular)

1. Open a secondary terminal window and navigate to the root directory of the frontend repository.
2. Install the required Node modules and reactive UI dependencies:

```
npm install
```

3. Boot the Angular development server:

```
ng serve
```

4. Upon successful compilation, open a web browser and navigate to `http://localhost:4200` to interact with the UI.

## 12 Lessons Learned

### 12.1 Successes and Positive Outcomes

- **Architectural Decoupling:** The strict separation between the Angular Single Page Application and the monolithic Spring Boot backend using a RESTful API ensured clean boundaries, enabling independent debugging and efficient parallel development.
- **Modern Frontend Reactivity:** Successfully implemented the modern **Angular Signals** API for core state management, which significantly simplified UI reactivity and optimized rendering performance compared to traditional RxJS observable patterns.
- **Data Modeling Integration:** Implementing the **JOINED** table inheritance strategy via Spring Data JPA proved highly effective for managing the complex multi-tier hierarchy (**CLIENT**, **MECHANIC**, **MANAGER**, **ADMIN**) without compromising relational database integrity.
- **Security Pipeline Implementation:** Building a robust, role-based access control (RBAC) system with stateless JWTs and rate-limiting filters successfully secured the application endpoints against unauthorized operational mutations.



## 12.2 Challenges Encountered

- **Concurrent Session Management:** Designing a mechanism to instantly invalidate distributed stateless tokens required an architectural compromise. Implementing the database-backed `tokenVersion` metric alongside the Angular `TabSyncService` (via the `BroadcastChannel` API) was logically complex but strictly necessary to prevent hijacked or stale sessions.
- **Environment Dependency Constraints:** Managing local execution friction, such as global Maven execution failures (Error 127) and IDE-specific reflection warnings regarding custom `env.properties` variables (e.g., `jwt.secret`), demonstrated the fragility of environment-dependent setups.

## 12.3 Future Improvements and Alternative Approaches

- **Containerization (Docker):** To completely eliminate the local setup issues documented in the installation phase (such as Maven wrapper requirements or manual PostgreSQL schema configurations), the entire application stack should be containerized. Providing a unified `docker-compose.yml` file would guarantee a seamless, reproducible deployment across any operating system.
- **Real-Time WebSocket Integration:** For future iterations, introducing bi-directional WebSocket communication (e.g., STOMP over SockJS) would enable instant server-to-client push notifications, keeping the Managerial workload grids and Client appointment statuses dynamically synchronized in real-time without relying on manual page refreshes.
- **Automated CI/CD Integration:** The current reliance on manual end-to-end testing and local script executions would benefit from an automated Continuous Integration/Continuous Deployment (CI/CD) pipeline (e.g., GitHub Actions) to systematically run the JaCoCo test suites upon every repository commit.